

3 - COMPONENT CONFIGURATION

Learning objectives

After completing this lab, you should be able to

- Add DUT virtual interfaces to driver
- Add configuration fields to driver
- Add physical device drivers to driver
- Add DUT virtual interfaces to reset sequence
- Add physical divide drivers to reset sequence
- Compile and simulate

Lab Duration:
60 Minutes

Getting Started

In Lab 2, you were able to generate stimulus. In this lab, you will add a physical interface to the driver and drive the stimulus through the DUT.

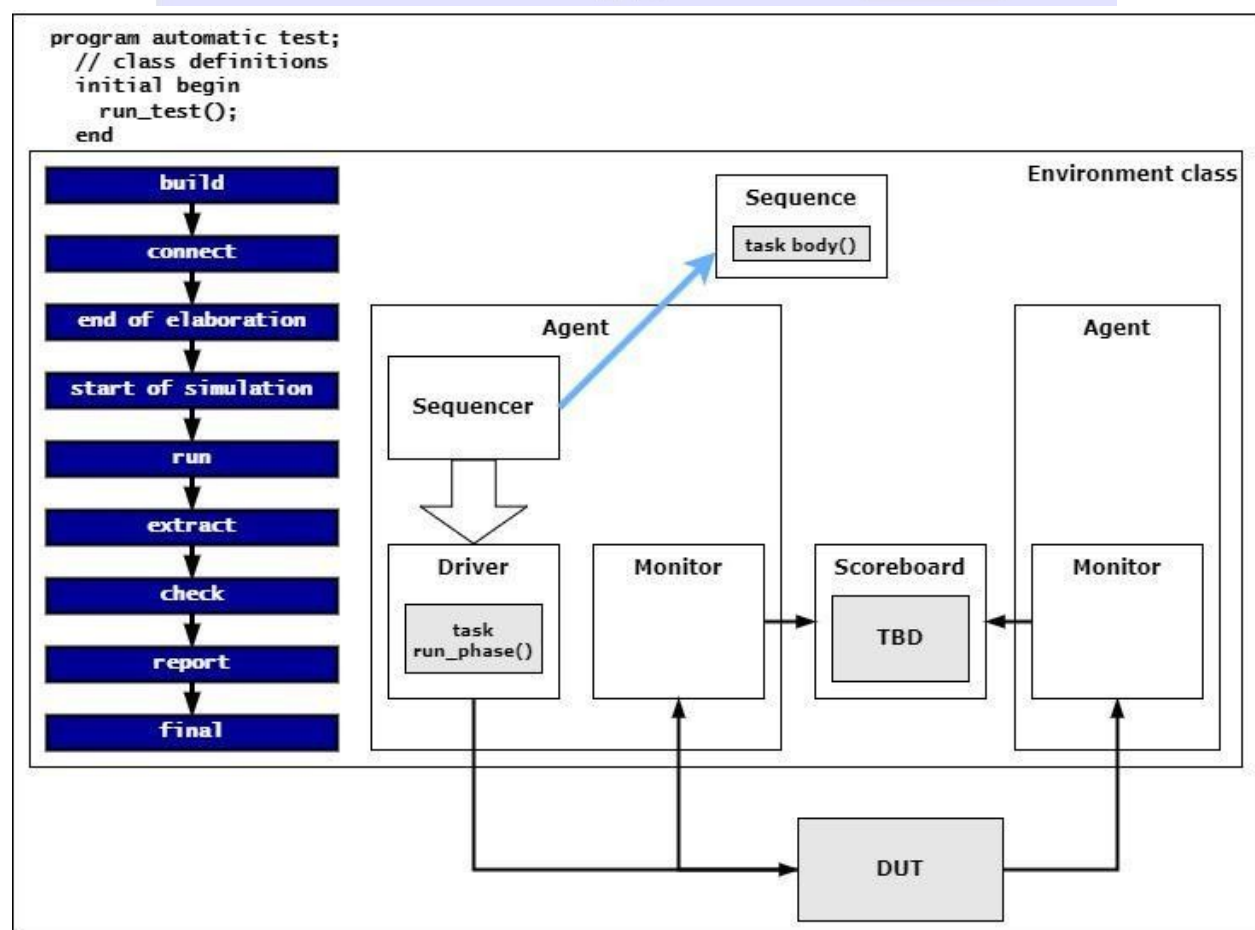
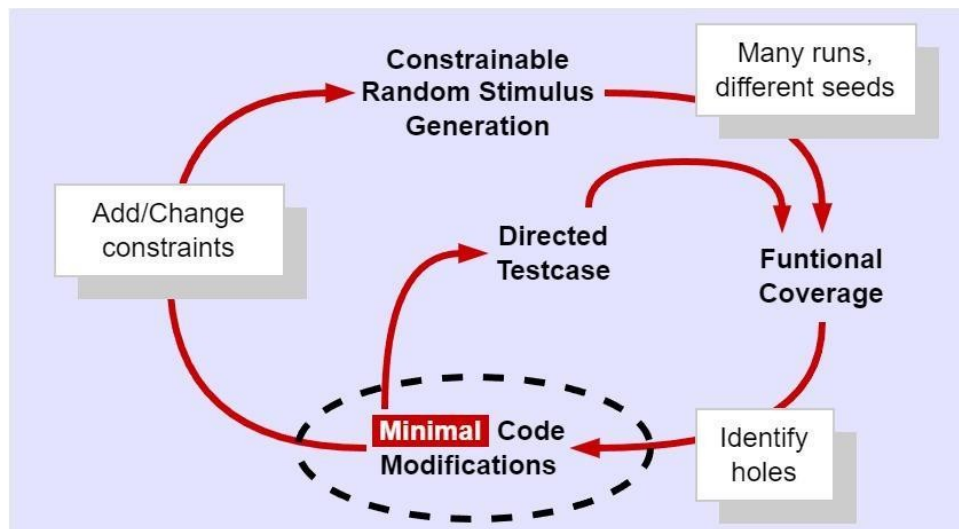


Figure 1. Lab 3 Testbench Architecture.

Lab Overview

The work flow for this lab is illustrated as follows.

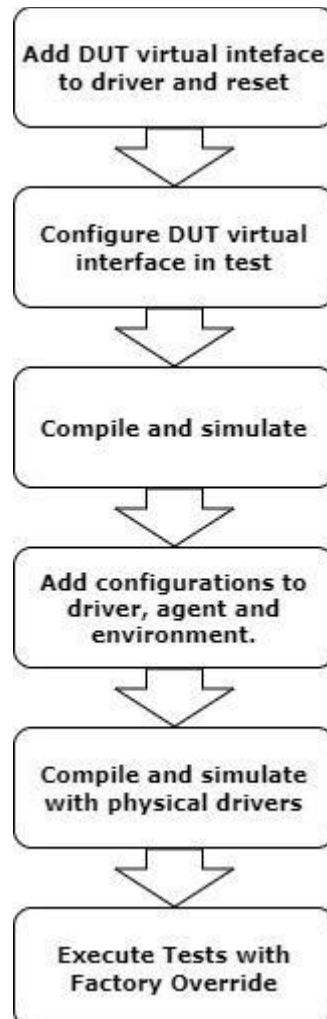


Figure 2. Lab 3 Flow Diagram

Implement Configurable Environment

Up to this point, you have not driven any test stimulus through the DUT. In order to do this, you need to embed and configure DUT virtual interfaces in physical drivers and implement the divide driver code.

In this lab, you will add the DUT virtual interface and other configuration fields to the driver, reset sequence, agent and environment. The device driver code will be written for you.

Task 1. Go into lab3 Working Directory

1. Open a new WSL terminal tab.

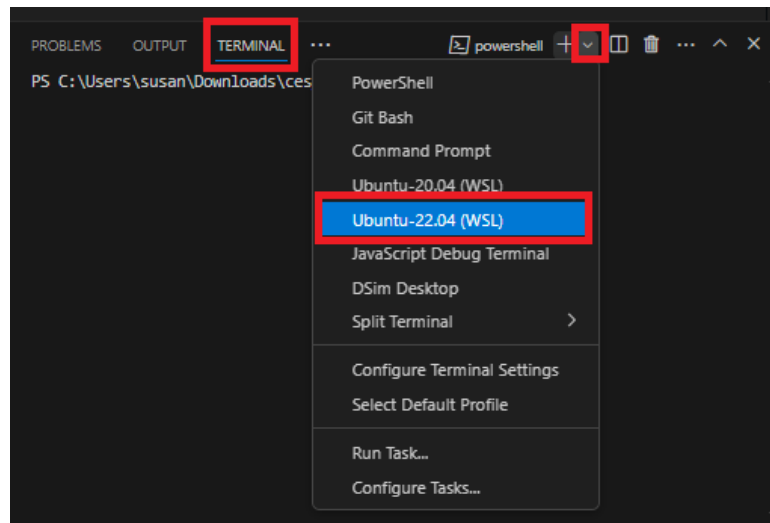


Figure 3. New WSL tab.

Go into **lab3** directory: **ces_uvm-1.2_2021.09/labs/lab3** directory.

Check the DSImDesktop, if no project has been loaded add the **my_labs_sim.dpf** as your project.

Go into **lab3/packages**, open the **router_stimulus_pkg.sv**, **router_env_pkg** and **router_and_pkg**. Add the **<labs/lab3/>** path to each include of these files, the same way it was done in lab2.

2. Into the **lab3** folder, rename the **test.sv** file as **test_3.sv** file.
3. Open the **test_3.sv** file with an editor.

4. Rename the **program automatic test** as **program automatic test_3**.

Include `<`include "uvm_macros.svh">` below `<import uvm_pkg::*;>` and save the file.

5. Open the **verif_filelist** file with an editor.
6. Delete the packages that have lab 2 in its name, add the following code lines and save the file:

```
rtl\router_test_top.sv
labs\lab1\test.sv
labs\lab2\test_2.sv
labs\lab3\packages\router_stimulus_pkg.sv
labs\lab3\packages\router_env_pkg.sv
labs\lab3\packages\router_test_pkg.sv
labs\lab3\test_3.sv
```

7. Hover over the **SIMULATION CONFIGURATION** drop-down menu and click the + button.

Fill and save the configuration tab with the name **lab3_sim**, and the next options:

```
-top work.test_3 -uvm 1.2 -L work +acc+b -waves
lab3_test.mxd -dump-agg -sv_seed random -timescale
1ns/1ps +UVM_TIMEOUT=1000000 +UVM_VERBOSITY=UVM_MEDIUM
+UVM_NO_DEPRECATED +UVM_OBJECT_MUST_HAVE_CONSTRUCTOR
+UVM_TESTNAME=test_base
```

Task 2. Add Interface and Configuration to Driver

1. Open **driver.sv** file in an editor
2. Look for : “// Lab 3 - Task 2, Step 2 and 3” and do the following:

Create a DUT virtual interface (**router_io**) handle, call it **vif**:

```
class driver extends uvm_driver #(packet);  
    virtual router_io vif;
```

3. Look for : “// Lab 3 - Task 2, Step 2 and 3” and do the following:

Add the ability to designate the driver to only drive a chosen port with an **int** property called **port_id** with the default value of **-1**:

```
int port_id = -1
```

This **port_id** is meant to configure the driver to only drive packets of matching source address (**sa**). If the incoming packet's address **sa** does not match the driver's **port_id**, that packet will be dropped.

If **port_id** is not set (**-1**), the driver will accept and drive all incoming packets. For this lab, you will leave the **port_id** at the default value of **-1**. In the next lab, you will make use of the **port_id**.

4. Look for : “// Lab 3 - Task 2, Step 4” and do the following:

Add **port_id** field to **uvm_component_utils**:

```
`uvm_component_utils_begin(driver)  
    `uvm_field(port_id, UVM_ALL_ON | UVM_DEC)  
`uvm_component_utils_end
```

Add _begin to macro

Notice that the virtual interface handle (**vif**) is not added to the **uvm_component_utils** macro list. This is because there is no support in the **uvm_component_utils** macro to accommodate virtual interfaces.

5. Look for : “// **Lab 3 - Task 2, Step 5**” and do the following:

Add the following code to retrieve the virtual interface in build phase:

```
function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);  
    uvm_config_db#(virtual router_io)::get(this, "", "vif", vif);  
endfunction: build_phase
```

Retrieve DUT virtual interface
and store handle in **vif**

Notice that the above code does not manually retrieve **port_id** field from the **config_db**. The reason is that properties listed via the **`uvm_field** macro is automatically retrieved from the **config_db** in the build phase of the base class. So, there is no need to retrieve the field manually.

The get function from the **uvm_config_db** returns a **1** when it finds the data that you're looking for, if not it returns a **0**. Due to this, it is a **good practice** to wrap this expression in a way that if a **0** is returned, a error is shown. Add the next code:

```
function void build_phase(uvm_phase phase);  
    super.build_phase(phase);  
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);  
    if (!(uvm_config_db#(virtual router_io)::get(this, "",  
    "vif", vif))) begin  
        `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Driver is  
not getting the IF from config DB"));  
end
```

Note: This practice can be replicated to other functions that have the same behavior (returning 0 or 1), helping to debug more easily in the future.

6. Look for : “// **Lab 3 - Task 2, Step 6**” and do the following:

Check the configuration properties in end_of elaboration phase:

```

function void end_of_elaboration_phase(uvm_phase phase);
    super.end_of_elaboration_phase(phase);

    if (!(port_id inside {-1, [0:15]})) begin
        `uvm_fatal("CFGERR", $sformatf("port_id must be {-1,
[0:15]}, not %0d!", port_id));
    end
    if (vif == null) begin
        `uvm_fatal("CFGERR", "Interface for Driver not set");
    end
end

```

7. Locate the **run_phase()** (Look for : “// Lab 3 - Task 2, Step 7”) method and add the following:

Check **port_id** to see if the driver should accept or drop the packet. If **port_id** is **-1**, or if **port_id** matches **req** object's **sa** field, call **send()** method to drive the content of the **req** object through the DUT. Otherwise, drop the **req** object without processing.

```

if (port_id inside { -1, req.sa }) begin
    send(req);
    `uvm_info("DRV_RUN", {"\n", req.sprint()}, UVM_MEDIUM);
end
seq_item_port.item_done();

```

To save lab time, the **start_of_simulation_phase()** method and the physical device driver methods are done for you.

8. Save and close the file.

Task 3. Agent Configuration

The agent contains the sequencer, the driver and the monitor. The agent should be treated by the higher layer structural components (environment, test) as a black box. This means that configuration of the sequencer, driver and monitor inside the agent should be done by the agent. The higher structural components should configure only the agent, not the individual components within the agent. The agent would then configure its own children components.

Since the configuration calls are the same as what you have already done, there is a no learning point in going through a typing exercise to enter the code in the agent. All the configuration code have been done for you in **input_agent.sv**.

1. Open **input_agent.sv** file in an editor
2. Look for the build_phase function.
3. As a good practice, add the next code to the get functions of the config_db:

```
if (!(uvm_config_db#(int)::get(this, "", "port_id",  
port_id))) begin  
    `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Input_Agent is  
not getting the IF from config DB"));  
end  
if (uvm_config_db#(int)::get(this, "", "port_id",  
port_id))
```

4. Save and close the file.

Task 4. Compile and simulate

1. Open **packet_sequence.sv** file in an editor
2. Go to line 117, inside the **uvm_do** macro do replace the next constraint:

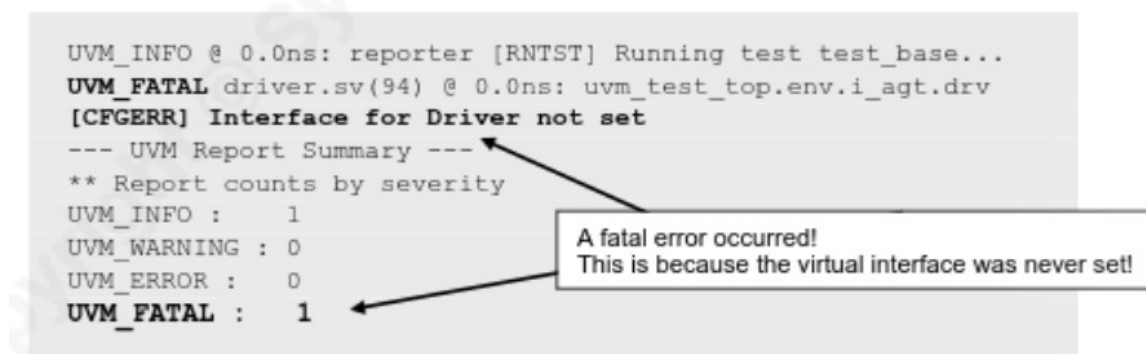
da inside valid_da;

with

da < valid_da.size(); da >= 0;

3. Save and close the file.
4. Compile and run the **lab3_sim** through the DSIM desktop.

You should see the following fatal error:



```
UVM_INFO @ 0.0ns: reporter [RNTST] Running test test_base...
UVM_FATAL driver.sv(94) @ 0.0ns: uvm_test_top.env.i_agt.drv
[CFGERR] Interface for Driver not set
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO : 1
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 1
```

A fatal error occurred!
This is because the virtual interface was never set!

Even though the DUT virtual interface was added in the driver, you have not yet configured it in the test. This is a fatal error that you must correct.

Task 5. Store Interfaces in Resource Database and Instantiation of the DUT

The interface of the DUT are static, the proper place to store the handles to these interfaces into the resource database is the program/module where `run_test()` is called. There are two interfaces that the test needs to access: to router port io and the router reset signal.

1. Open **test_3.sv** file in an editor.
2. Now it is needed to instantiate the DUT that will be stimulated and the clock that it needs to work. Add the next code:

```
parameter simulation_cycle = 100 ;
bit   SystemClock;

reset_io  reset_if(SystemClock);
router_io router_if(SystemClock);
host_io   host_if(SystemClock);
router
dut(.clk(SystemClock), .reset_n(reset_if.reset_n), .io(router_i
f), .host(host_if));
```

3. Look for : “// **Lab 3 - Task 5, Step 2**” and do the following:
 - Populate the resource database with the interfaces needed by the test. Be careful, notice that our interfaces are just **router_if** and **reset_if**, no **router_test_top.router_if** neither **router_test_top.reset_if**
3. Save and close the file.

Task 6. Configuration Agents in Test

The proper place to configure the physical agents to use these interfaces is in the base test.

1. Open **test_collection.sv** file in an editor.
2. Look for : “// **Lab 3 - Task 6, Step 2**” and do the following:

Declare the two virtual interface handles to the interfaces

```
class test_base extends uvm_test;
  `uvm_component_utils(test_base)

  router_env env;

  virtual router_io router_vif;
  virtual reset_io reset_vif;
```

3. Locate the **build_phase** method in **test_base** class.
4. Look for : “// **Lab 3 - Task 6, Step 4**” and do the following:
 - Retrieve the interfaces from resource database. Remember the use of good practices.
5. Look for : “// **Lab 3 - Task 6, Step 5**” and do the following:
 - Then configure the agent with these interfaces
6. Save and compile the file

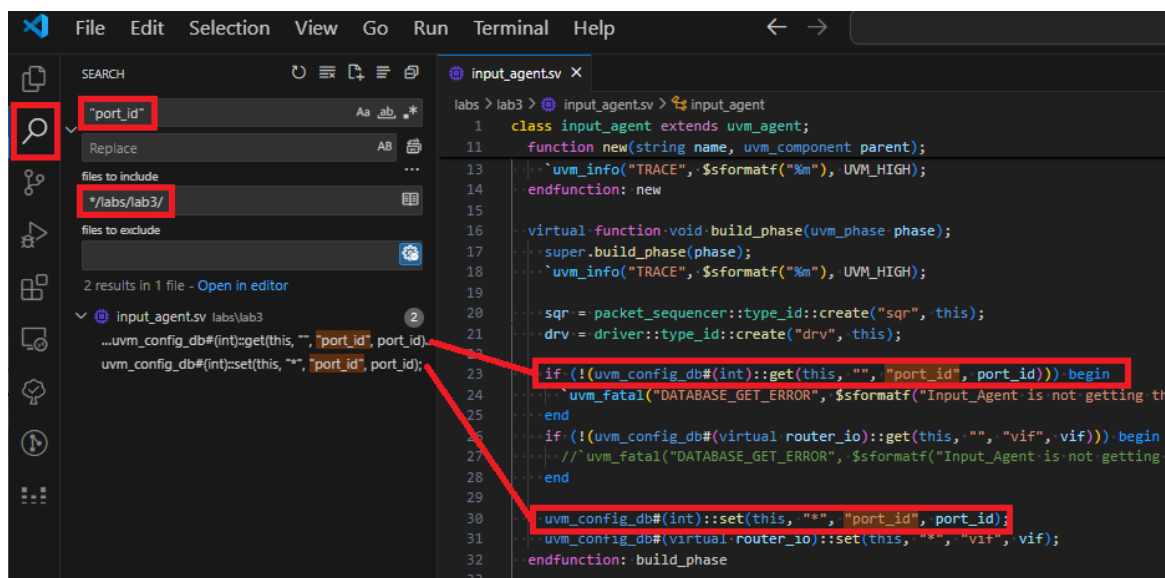
Task 7. Compile And Debug The Program

1. Recompile and run the **lab3_sim** through the DSIM desktop.

The next fatal error should appear:

```
UVM_FATAL labs\lab3\input_agent.sv(24) @ 0.0ns:
uvm_test_top.env.i_agt [DATABASE_GET_ERROR] Input_Agent is not
getting the port_id from config DB
```

Go to the Search tab of VSC and search for "**port_id**" just in the ***/labs/lab3/** directory.



As you can observe, "**port_id**" appears only two times in **lab3** directory. Whats wrong?

If you analyze it, the code is trying to get "**port_id**" when it has never being set before, so the **get** function is returning a **0** value. In more complex environments, the use of good practices become more important.

In a case like this one, if a **get** error is shown you can search for the places where the **set** function are done and check for syntax mismatching or if the **set** function is in the place.

2. Open **input_agent.sv** file in an editor.
3. This time, for avoiding this error, comment the fatal error related to "**port_id**":

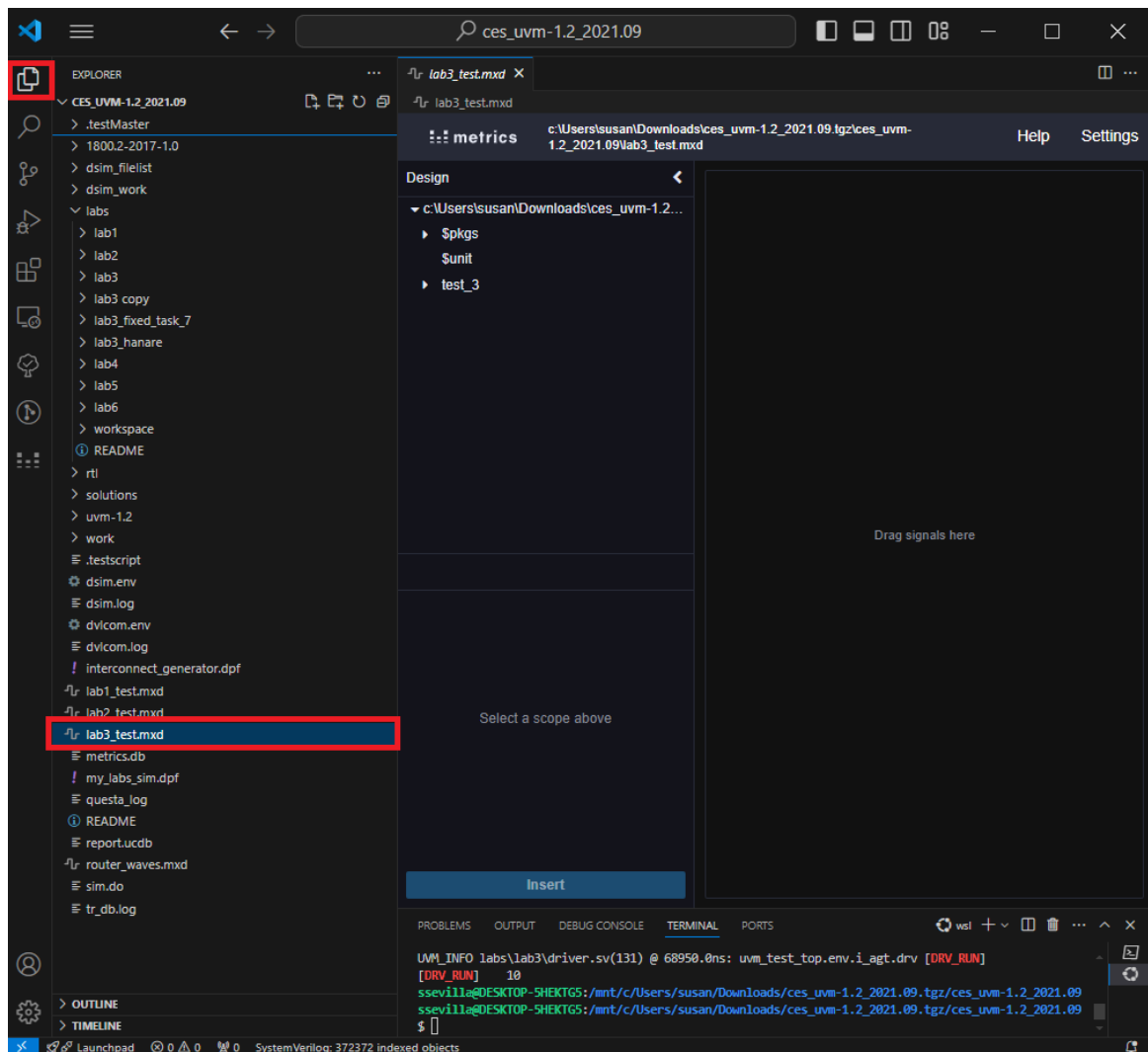
```
if (!(uvm_config_db#(int)::get(this, "", "port_id", port_id))) begin
    // `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Input_Agent is not getting
    the port_id from config DB"));
end
```

4. Save and close the file.
5. Recompile and run the **lab3_sim** through the DSIM desktop.

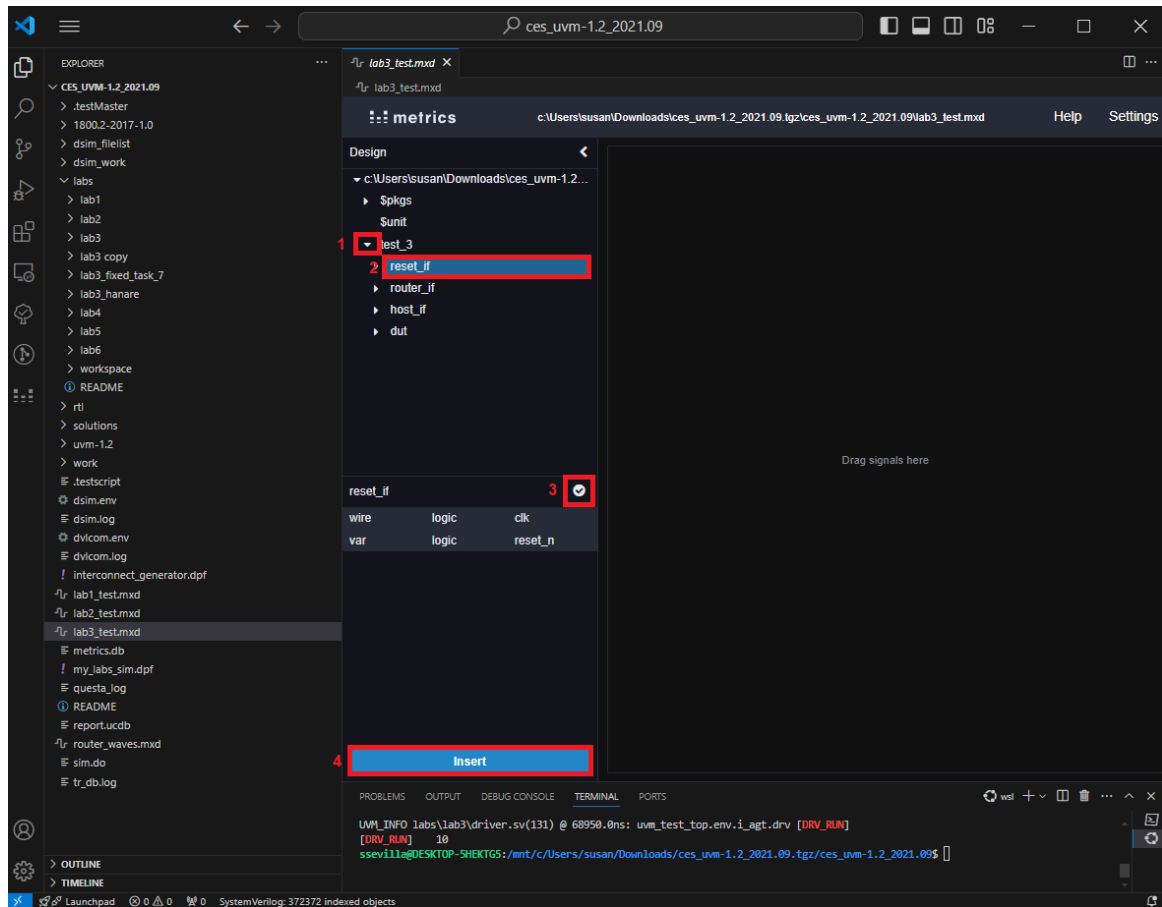
How many packets did the driver process? (Hint: use grep command in the WSL terminal window and check the **DRV_RUN** counter)

But, did these packets propagate correctly through the DUT?

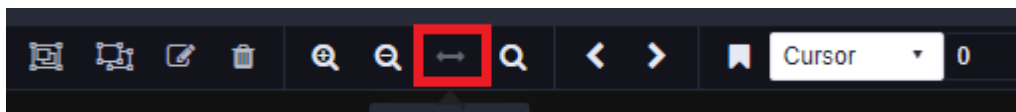
6. Go to the VSC Explorer tab and look for the **lab3_test.mxd** file, double click it. A DSIm wave visualizer should be open as a tab.



7. Insert the reset interface signals by following the steps that are shown in the next image.



8. Repeat the process for inserting the **router_if** signals.
9. Use the option **Zoom to fit**.



Now you should see that all outputs values are red (unknown)! This is because the DUT must be reset before it can successfully process inputs.

10. Close the waves tab.

Task 8. Generate Reset Transactions

In this task, you will develop the **reset_sequence** to perform a DUT reset.

1. Open the **reset_sequence.sv** file in an editor.
2. In **body()** task
 - De-assert reset for 2 cycles
 - Then, assert reset for 1 cycle
 - Followed by de-assert for 15 cycles.

```
virtual task body();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    `ifndef UVM_VERSION
        `uvm_do_with(req, {kind == DEASSERT; cycles == 2;});
        `uvm_do_with(req, {kind == ASSERT; cycles == 1;});
        `uvm_do_with(req, {kind == DEASSERT; cycles == 15;});

        // For IEEE UVM

    `else
        `uvm_do(req,, {kind == DEASSERT; cycles == 2;});
        `uvm_do(req,, {kind == ASSERT; cycles == 1;});
        `uvm_do(req,, {kind == DEASSERT; cycles == 15;});
    `endif

endtask: bodv
```

3. Save and close the file.

To save lab time, this file is included in the router stimulus package for you.

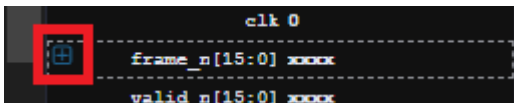
Task 9. Compile and Simulate with Reset Sequence

1. Recompile and run the **lab3_sim** through the DSIM desktop.

Did it work? Check it in the waveform.

2. Go to the VSC Explorer tab and look for the **lab3_test.mxd** file, double click it. A DSIm wave visualizer should be open as a tab.
3. Insert the **reset_if** signals.
4. Insert the **router_if** signals.
5. Use the option **Zoom to fit**.

You should now see that the output signals are no longer red. However, the sequence was designed to send 10 packets through the DUT. In the waveform window, this is not what you are seeing (you may need to adjust the waveform window to see all the 16 frame signals). There is still a problem.



6. Close the waves tab.

Task 10. Control Signal De-Assertion Sequence

With the reset agent and reset sequence, the only thing that happened was the assertion and de-assertion of the reset signal. How about the control signals of the interfaces? The reset agent and the reset sequence should not be responsible for taking care of all the signals of the DUT. Doing so complicates the development of individual tests and integration.

There are two ways to resolve this problem. One, create and configure a sequence to de_assert the control signals. Or, two, de-assert the control signals within the driver's **reset_phase()** method.

For flexible controls of reset priorities, you should de-assert the control signals with sequences. De-asserting the control signals in the driver's **reset_phase()** is discouraged.

A sequence to de_assert the router input control signals has been done for you in **router_input_port_reset_sequence.sv**

Take a look at the code if you are interested. Otherwise, configure this sequence to be executed at the reset_phase.

1. Open the **router_env.sv** file in an editor.
2. Look for : "**Lab 3 - Task 10, Step 2**" and do the following:
 - Configure the inputs agent's (**i_agt**) sequencer (**sqr**) to execute the **router_input_port_reset_sequence** at the **reset_phase**.
3. Save and close the file.

Task 11. Compile & Simulate with Driver Reset Phases

-
1. Recompile and run the **lab3_sim** through the DSIM desktop.
 2. Go to the VSC Explorer tab and look for the **lab3_test.mxd** file, double click it. A DSIM wave visualizer should be open as a tab.
 3. Insert the **reset_if** signals.
 4. Insert the **router_if** signals.
 5. Use the option **Zoom to fit**.

You should see the correct behavior: 10 packets came out of the DUT.

A test has been written for you that executes a sequence which sets the da to 3 and generates 20 packets.

6. Close the waves tab.
7. Hover over the lab3_sim from the **SIMULATION CONFIGURATION** click right click it, choose "*Configure Simulation*".

Replace **+UVM_TESTNAME=test_base** with **+UVM_TESTNAME=test_da_3_seq** and save the changes.

Run the **lab3_sim** through the DSIM desktop.

8. Take a look at the **lab3_test.mxd** file, plot the signals. You should see the 20 packets came out of output port 3.
9. Close the waves tab.

Solutions

test_3.sv

```

program automatic test_3;
import uvm_pkg::*;
`include "uvm_macros.svh"
import router_test_pkg::*;

    parameter simulation_cycle = 100 ;
    bit  SystemClock;

    reset_io  reset_if(SystemClock);
    router_io router_if(SystemClock);
    host_io   host_if(SystemClock);
    router

dut(.clk(SystemClock), .reset_n(reset_if.reset_n), .io(router_if), .host(host
_if));

    initial begin
        forever #(simulation_cycle/2) SystemClock = ~SystemClock ;
    end

    initial begin
        uvm_resource_db#(virtual router_io)::set("router_vif", "", router_if);
        uvm_resource_db#(virtual reset_io)::set("reset_vif", "", reset_if);

        $timeformat(-9, 1, "ns", 10);
        run_test();
    end

endprogram

```

test_collection.sv

```

class test_base extends uvm_test;
  `uvm_component_utils(test_base)

  router_env env;
  virtual router_io router_vif;
  virtual reset_io reset_vif;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction: new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    env = router_env::type_id::create("env", this);

    if (!(uvm_resource_db#(virtual router_io)::read_by_type("router_vif",
router_vif, this))) begin
      `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Test is not getting the IF from
config DB"));
    end
    if (!(uvm_resource_db#(virtual reset_io)::read_by_type("reset_vif", reset_vif,
this))) begin
      `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Test is not getting the IF from
config DB"));
    end

    uvm_config_db#(virtual router_io)::set(this, "env.i_agt", "vif", router_vif);
    uvm_config_db#(virtual reset_io)::set(this, "env.r_agt", "vif", reset_vif);
  endfunction: build_phase

  virtual function void final_phase(uvm_phase phase);
    super.final_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO, "TOPOLOGY")) begin
      uvm_root::get().print_topology();
    end

    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO, "FACTORY")) begin
      uvm_factory::get().print();
    end
  end

```

```

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
        set_inst_override_by_type("env.i_agt.sqr.*", packet::get_type(),
packet_da_3::get_type());
    endfunction: build_phase
endclass: test_da_3_inst

class test_da_3_type extends test_base;
    `uvm_component_utils(test_da_3_type)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    endfunction: new

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
        set_type_override_by_type(packet::get_type(), packet_da_3::get_type());
    endfunction: build_phase
endclass: test_da_3_type

class test_da_3_seq extends test_base;
    `uvm_component_utils(test_da_3_seq)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    endfunction: new

    virtual function void build_phase(uvm_phase phase);
        packet_sequence::int_q_t valid_da = {3};
        super.build_phase(phase);

```


router_env.sv

```
class router_env extends uvm_env;
  input_agent i_agt;
  reset_agent r_agt;

  `uvm_component_utils(router_env)

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction: new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    i_agt = input_agent::type_id::create("i_agt", this);
    uvm_config_db #(uvm_object_wrapper)::set(this, "i_agt.sqr.main_phase",
"default_sequence", packet_sequence::get_type());

    uvm_config_db #(uvm_object_wrapper)::set(this, "i_agt.sqr.reset_phase",
"default_sequence", router_input_port_reset_sequence::get_type());

    // In the interest of minimizing lab time, the reset agent has been
    // instantiated and created for you. The reset sequence execution is also
    // done for you.
    r_agt = reset_agent::type_id::create("r_agt", this);
    uvm_config_db #(uvm_object_wrapper)::set(this, "r_agt.sqr.reset_phase",
```

driver.sv

```
class driver extends uvm_driver #(packet);
    virtual router_io vif;           // DUT virtual interface
    int port_id = -1;               // Driver's designated port

    `uvm_component_utils_begin(driver)
        `uvm_field_int(port_id, UVM_ALL_ON | UVM_DEC)
    `uvm_component_utils_end

    function new(string name, uvm_component parent);
        super.new(name, parent);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    endfunction: new

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
        if (!(uvm_config_db#(virtual router_io)::get(this, "", "vif", vif))) begin
            `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Driver is not getting the IF from
config DB"));
        end
    endfunction: build_phase

    // The UVM end_of_elaboration phase is designed for checking the
    // correctness of testbench structure and the configuration variables.
    function void end_of_elaboration_phase(uvm_phase phase);
        super.end_of_elaboration_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
        if (!(port_id inside {-1, [0:15]})) begin
            `uvm_fatal("CFGERR", $sformatf("port_id must be {-1, [0:15]}, not %0d!",
port_id));
        end
        if (vif == null) begin
            `uvm_fatal("CFGERR", "Interface for Driver not set");
        end
    endfunction: end_of_elaboration_phase

    // The UVM start_of_simulation phase is designed for displaying the testbench
    configuration
    // before any active verification operation starts.
    // For the sake of lab time, the start_of_simulation method is done for you.
    virtual function void start_of_simulation_phase(uvm_phase phase);
        super.start_of_simulation_phase(phase);
        `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
        `uvm_info("DRV_CFG", $sformatf("port_id is: %0d", port_id), UVM_MEDIUM);
    endfunction: start_of_simulation_phase

    virtual task run_phase(uvm_phase phase);
```

```

// In the interest of lab time, all device drivers have been done for you:
virtual task send(packet tr);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    send_address(tr);
    send_pad(tr);
    send_payload(tr);
endtask: send

virtual task send_address(packet tr);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    vif.drvClk.frame_n[tr.sa] <= 1'b0;
    for(int i=0; i<4; i++) begin
        vif.drvClk.din[tr.sa] <= tr.da[i];
        @(vif.drvClk);
    end
endtask: send_address

virtual task send_pad(packet tr);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    vif.drvClk.din[tr.sa] <= 1'b1;
    vif.drvClk.valid_n[tr.sa] <= 1'b1;
    repeat(5) @(vif.drvClk);
endtask: send_pad

virtual task send_payload(packet tr);
    logic [7:0] datum;
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    while(!vif.drvClk.busy_n[tr.sa]) @(vif.drvClk);
    foreach(tr.payload[index]) begin
        datum = tr.payload[index];
        for(int i=0; i<$size(tr.payload, 2); i++) begin
            vif.drvClk.din[tr.sa] <= datum[i];
            vif.drvClk.valid_n[tr.sa] <= 1'b0;
            vif.drvClk.frame_n[tr.sa] <= ((tr.payload.size()-1) == index) && (i==7);
            @(vif.drvClk);
        end
    end
end

```

```

class input_agent extends uvm_agent;
  `uvm_component_utils(input_agent)

  typedef uvm_sequencer #(packet) packet_sequencer;

  virtual router_io vif;
  packet_sequencer  sqr;
  driver            drv;
  int               port_id = -1;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction: new

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    sqr = packet_sequencer::type_id::create("sqr", this);
    drv = driver::type_id::create("drv", this);

    if (!(uvm_config_db#(int)::get(this, "", "port_id", port_id))) begin
      //`uvm_fatal("DATABASE_GET_ERROR", $sformatf("Input_Agent is not
getting the port_id from config DB"));
    end
    if (!(uvm_config_db#(virtual router_io)::get(this, "", "vif", vif)))
begin
  `uvm_fatal("DATABASE_GET_ERROR", $sformatf("Input_Agent is not getting
the IF from config DB"));
end

    uvm_config_db#(int)::set(this, "", "port_id", port_id);
    uvm_config_db#(virtual router_io)::set(this, "", "vif", vif);
  endfunction: build_phase

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    drv.seq_item_port.connect(sqr.seq_item_export);
  endfunction: connect_phase

```

router_input_port_reset_sequence.sv

```

class router_input_port_reset_sequence extends uvm_sequence #(packet);
  virtual router_io vif;           // DUT virtual interface
  int               port_id = -1;  // Driver's designated port

  `uvm_object_utils_begin(router_input_port_reset_sequence)
    `uvm_field_int(port_id, UVM_DEFAULT | UVM_DEC)
  `uvm_component_utils_end

```

```

// Lab 3
//
// The the agent associated with an interface is fully responsible for
setting the
// the control signals to the default value when reset is applied.
//
// For this lab, we are not using the port_id because this lab is still
just a part
// of initial bringup process. There is only one agent in the environment.
// So, if port_id is -1 (not using port_id), this sequence will reset the
control
// signals for all router (DUT) ports. In the next lab, when you implement
a
// dedicated agent for each port, the port_id will be set. This sequence
will then
// only reset the control signals of the designated port.
//
virtual task body();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    if (port_id == -1) begin
        vif.frame_n = '1;
        vif.valid_n = '1;
        // ...
    end
endtask

```

```

// Reset structure is done in the same way as all other UVM component
// structure.
// A reset agent contains a reset sequencer, driver and monitor.
//
// The reset sequencer executes a reset sequence (this file), and passes the
// reset
// transaction (reset_tr) to the reset driver. The reset driver then
// assert/de-assert
// the reset signal as specified in the reset transaction.
//
// Within the reset transaction class, there is a control command field
// called kind.
// If the kind field is ASSERT, then the driver will assert the reset signal
// for
// the number of clock cycles are specified in the cycles field. Similar
// action
// takes place for the DEASSERT command.
//
// For example, if the reset signal need to be asserted for 1 cycles then de-
// asserted
// for 15 clock cycles, the potential code might look like:
//
// reset_tr tr = reset_tr::type_id::create("tr");
// tr.randomize() with {kind == ASSERT; cycles == 1;};
// tr.randomize() with {kind == DEASSERT; cycles == 15;};

class reset_tr extends uvm_sequence_item;
    typedef enum {ASSERT, DEASSERT} kind_e;
    rand kind_e kind;
    rand int unsigned cycles = 1;

```

reset_sequence.sv

```

class reset_sequence extends uvm_sequence #(reset_tr);
  `uvm_object_utils(reset_sequence)

  function new(string name = "reset_sequence");
    super.new(name);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    `ifndef UVM_VERSION_1_1
      set_automatic_phase_objection(1);
    `endif
  endfunction: new

  virtual task body();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);

    `ifndef UVM_VERSION
      `uvm_do_with(req, {kind == DEASSERT; cycles == 2;});
      `uvm_do_with(req, {kind == ASSERT; cycles == 1;});
      `uvm_do_with(req, {kind == DEASSERT; cycles == 15;});
    `else // For IEEE UVM
      `uvm_do(req,,, {kind == DEASSERT; cycles == 2;});
      `uvm_do(req,,, {kind == ASSERT; cycles == 1;});
      `uvm_do(req,,, {kind == DEASSERT; cycles == 15;});
    `endif
  endtask: body

  `ifdef UVM_VERSION_1_1
  virtual task pre_start();
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    if ((get_parent_sequence() == null) && (starting_phase != null)) begin
      starting_phase.raise_objection(this);
    end
  endtask: pre_start

  virtual task post_start():

```